Consultas sobre columnas JSONB

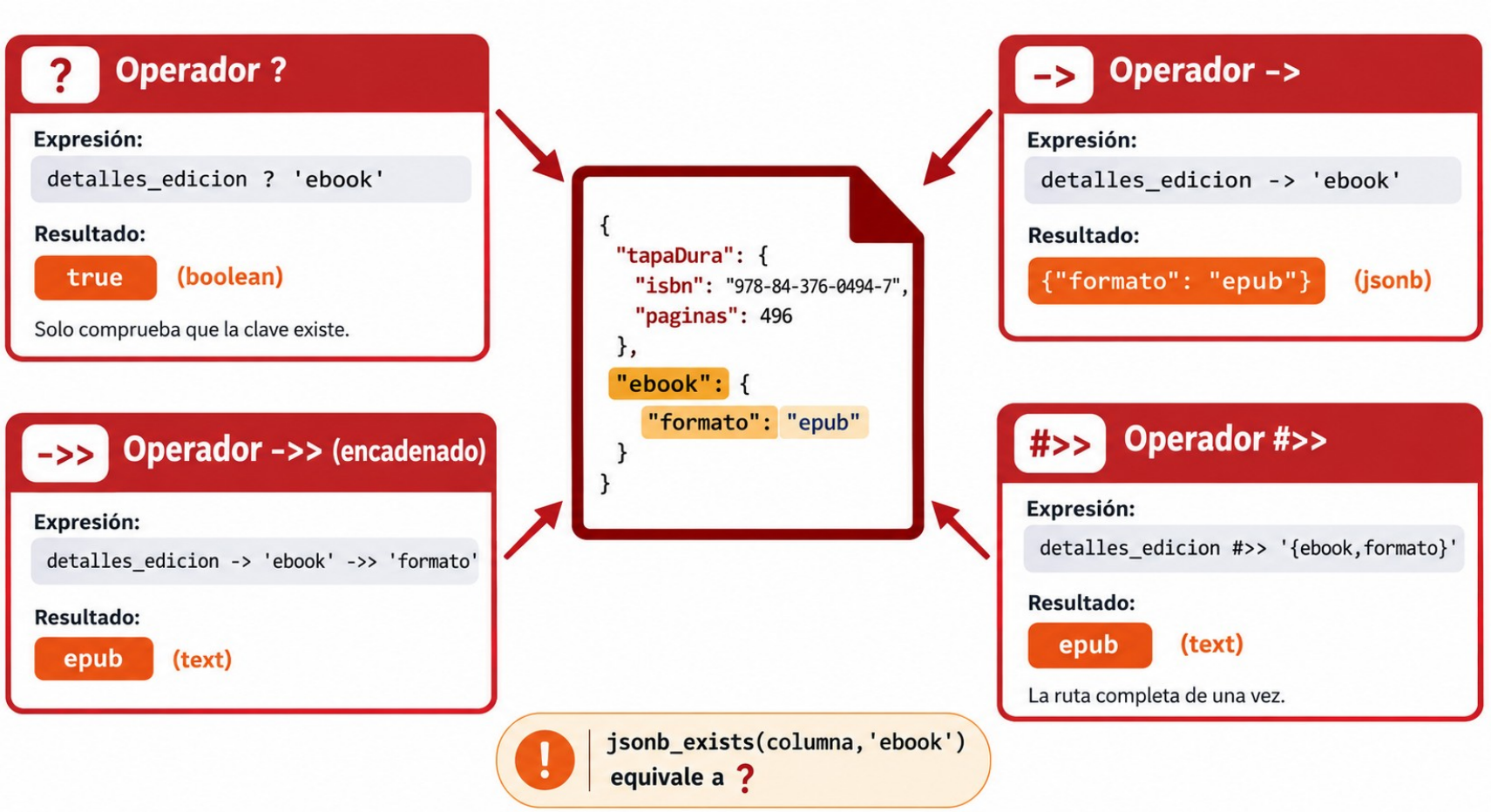
2. Consultas sobre columnas JSONB

Consultas sobre columnas JSONB

Ya sabes guardar datos estructurados en una columna JSONB. El siguiente paso es filtrar por su contenido. Para ello no bastan los comparadores habituales sobre valores escalares: necesitas operadores y funciones de PostgreSQL capaces de acceder a las claves y valores almacenados dentro del JSON.

Consultar dentro de un JSON: operadores y funciones

Con una columna normal, un WHERE compara el valor completo. Con JSONB hay que comprobar si existe (o qué vale) una clave concreta dentro del objeto: SQL puro, directamente contra PostgreSQL.



Un ejemplo completo: disponibleEnFormato

Un filtro más en el buscador: "solo libros disponibles en ebook". El dato vive dentro de la columna JSONB detallesEdicion.

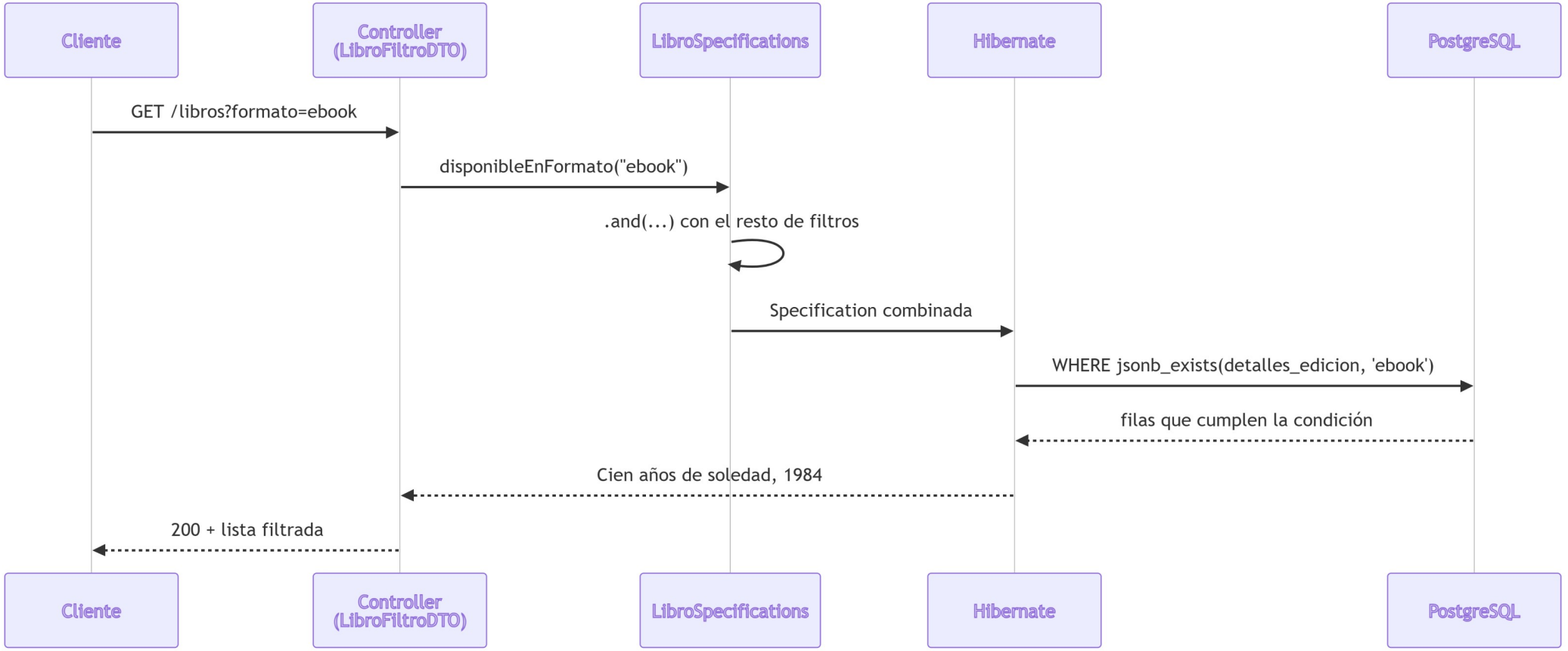
id	titulo	detalles_edicion
1	Cien años de soledad	{"tapaDura":{...}, "ebook":{"formato":"epub"}}
2	El nombre del viento	{"bolsillo":{...}}
3	Rayuela	{"tapaDura":{...}}
4	1984	{"ebook":{"formato":"mobi"}, "bolsillo":{...}}

```
SELECT titulo FROM libro WHERE detalles_edicion ? 'ebook';

titulo
-----------------------
Cien años de soledad
1984
```

Es la respuesta que debe dar la API cuando alguien busque ?formato=ebook.

De la petición HTTP a las dos filas



El parámetro de consulta, el DTO, la Specification, .and(...) ya son conocidos — la novedad es el tramo Hibernate→PostgreSQL: cómo jsonb_exists(...) llega a formar parte de la consulta generada.

La Specification, con jsonb_exists

```
public static Specification<Libro> disponibleEnFormato(String formato) {
    if (formato == null || formato.isBlank()) {
        return Specification.unrestricted();
    }

    return (root, query, criteriaBuilder) ->
            criteriaBuilder.isTrue(
                    criteriaBuilder.function(
                            "jsonb_exists",
                            Boolean.class,
                            root.get("detallesEdicion"),
                            criteriaBuilder.literal(formato)
                    )
            );
}
```

tituloContiene o precioMayorOIgualA usan métodos directos de criteriaBuilder sobre columnas de tipo simple. Aquí no existe un método para "existe esta clave en este JSON": es una función específica de PostgreSQL, no del estándar JPA. Por eso hace falta criteriaBuilder.function(...): nombre de la función, tipo de retorno y argumentos.

Combinada con el resto, de forma transparente

```
Specification<Libro> spec = Specification
        .where(LibroSpecifications.tituloContiene(filtro.titulo()))
        .and(LibroSpecifications.precioMayorOIgualA(filtro.precioMin()))
        .and(LibroSpecifications.disponibleEnFormato(filtro.formato()));
```

Filtrar por JSONB o por una columna relacional corriente es exactamente lo mismo desde fuera — una Specification más, encadenada con .and(...). Toda la complejidad queda encapsulada dentro de disponibleEnFormato.

Y si la clave está más adentro

jsonb_exists/? solo comprueban si una clave de primer nivel existe. "¿Tiene ebook?" no es lo mismo que "¿el ebook está en formato mobi?": para esa pregunta más concreta hace falta bajar un nivel:

```
-- Encadenando -> hasta el último nivel, y ->> para el valor final
SELECT titulo FROM libro
WHERE detalles_edicion -> 'ebook' ->> 'formato' = 'mobi';

-- Equivalente con el operador de ruta #>>
SELECT titulo FROM libro
WHERE detalles_edicion #>> '{ebook,formato}' = 'mobi';
```

El resultado ahora es solo 1984 — jsonb_exists(..., 'ebook') no distinguiría entre los dos libros, porque nunca mira el valor de la clave, solo si está presente.

Existe la clave no es lo mismo que su valor



Desde Criteria API: jsonb_extract_path_text

```
criteriaBuilder.equal(
        criteriaBuilder.function(
                "jsonb_extract_path_text",
                String.class,
                root.get("detallesEdicion"),
                criteriaBuilder.literal("ebook"),
                criteriaBuilder.literal("formato")
        ),
        "mobi"
);
```

La técnica es idéntica a la de disponibleEnFormato: nombre de la función nativa, tipo de retorno, argumentos — solo cambia la función y qué se hace con el resultado.

Qué consultas JSONB acelera el índice GIN

Tener un índice GIN no significa que toda consulta JSONB sea rápida

Consulta	Índice GIN sobre detalles_edicion	Por qué
`WHERE detalles_edicion ? 'ebook'`	 Puede usarlo	El operador ? y también @> están pensados para GIN (si el planificador estima que compensa).
`WHERE jsonb_exists(detalles_edicion, 'ebook')`	 No lo des por hecho	Mismo resultado lógico, pero PostgreSQL documenta el operador ?, no la llamada directa a la función.
`WHERE detalles_edicion -> 'ebook' ->> 'formato' = 'mobi'`	 No lo aprovecha	Extrae un valor como texto: haría falta un índice de expresión sobre (`detalles_edicion -> 'ebook' ->> 'formato'`).

La alternativa: @Query con SQL nativo

Retoma el Tema 1: cuando JPQL no llegaba a algo, la solución era @Query(nativeQuery = true). jsonb_exists y ? no existen en JPQL, que solo conoce el vocabulario común de JPA.

```
public interface LibroRepository extends JpaRepository<Libro, Long> {

    @Query(value = "SELECT * FROM libro WHERE jsonb_exists(detalles_edicion, :formato)",
           nativeQuery = true)
    List<Libro> buscarPorFormato(@Param("formato") String formato);
}
```

¿Por qué jsonb_exists y no ?: JDBC y JPA usan el carácter ? para los parámetros posicionales, y según la versión el operador JSONB ? puede confundirse con uno de esos marcadores. La función expresa la misma comprobación sin ese carácter.

A diferencia del ranking con ROW_NUMBER() del Tema 1, SELECT * devuelve las columnas de Libro: Spring Data lo mapea a List<Libro> sin problema.

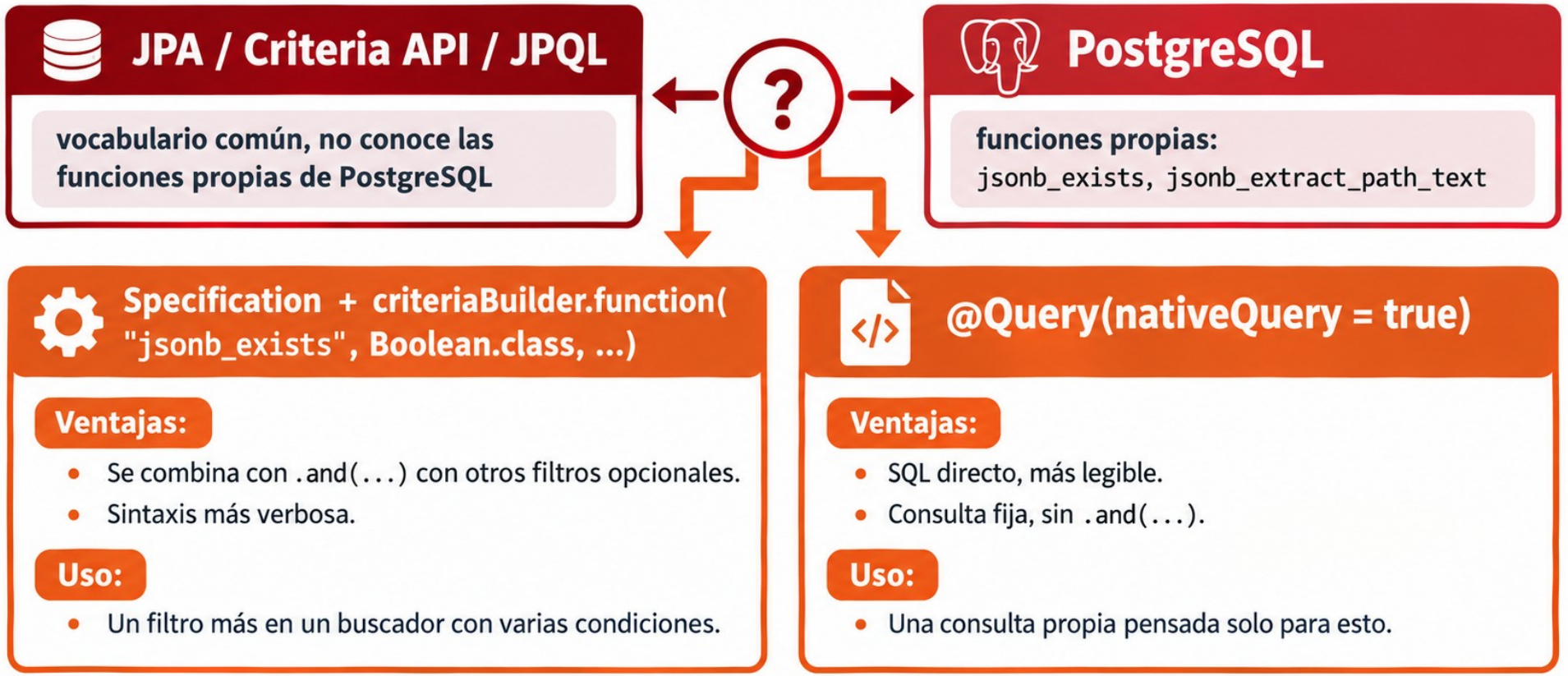
La misma alternativa para la ruta anidada

```
@Query(value = "SELECT * FROM libro WHERE detalles_edicion -> 'ebook' ->> 'formato' = :formato",
       nativeQuery = true)
List<Libro> buscarPorFormatoEbook(@Param("formato") String formato);
```

buscarPorFormatoEbook("mobi") devuelve solo 1984, el mismo resultado que con jsonb_extract_path_text desde Criteria API. Como aquí no interviene el operador ?, no hay conflicto con el marcador de parámetro de JDBC: puedes escribir la ruta con -> y ->> tal cual, como en psql.

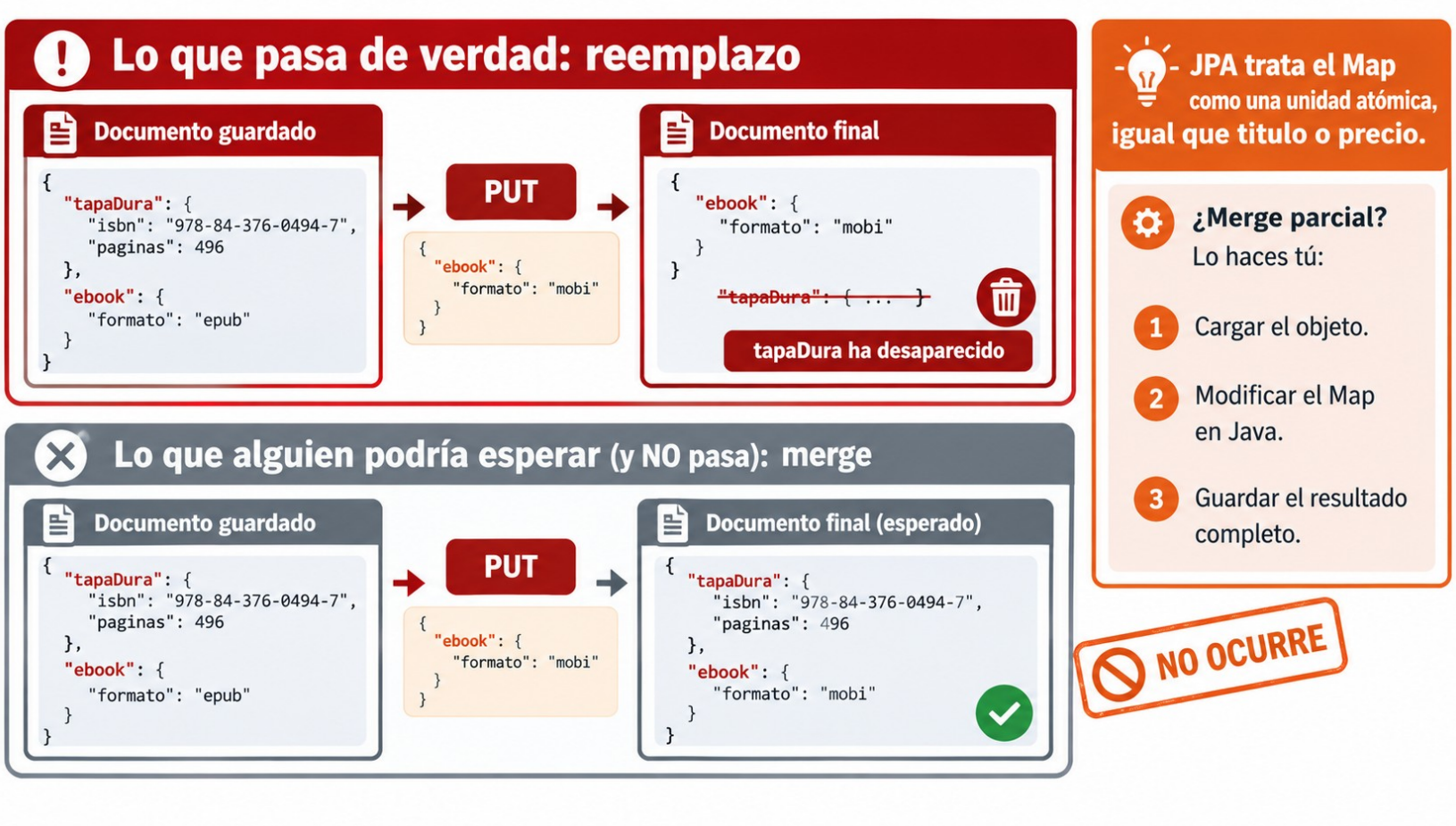
¿Entonces criteriaBuilder.function(...) o @Query nativa? La misma pregunta del Tema 1 entre Specifications y @Query con JPQL, con la misma respuesta (siguiente diapositiva).

Specification con function o @Query nativa



En @Query se usa `jsonb_exists` y no `?` porque `?` choca con el marcador de parámetros de JDBC.

Modificar JSONB: reemplazo, no merge




```
libro.setDetallesEdicion(dto.detallesEdicion()); // sustituye el Map entero
```

Ideas clave

1. jsonb_exists(columna, clave) (o el operador ?) comprueba si una clave existe; -> y ->> extraen valores (como JSON o como texto).

2. El índice GIN puede acelerar operadores como ?. jsonb_exists(...) hace la misma comprobación, pero no debe darse por hecho que use el índice igual.

3. Para comparar un valor anidado se encadena -> / ->> o se usa #>>: ahí el índice GIN de ? ya no ayuda igual.

4. Consultar JSONB desde Criteria API requiere criteriaBuilder.function(...): son funciones del motor, no del estándar JPA.

Ideas clave (cont.)

1. @Query(nativeQuery = true) es más directa cuando la consulta JSONB es fija. Se usa jsonb_exists(...) para evitar la ambigüedad entre ? y los parámetros de JDBC/JPA.

2. Combinada con .and(...), una Specification sobre JSONB se usa igual que una sobre una columna normal.

3. Actualizar un campo JSONB reemplaza el objeto completo: JPA no hace merge parcial. Si hace falta, se hace en memoria antes de guardar.

4. @Transactional no cambia en nada por tener columnas JSONB de por medio.